Linearizing Temporal Graph Dynamics with State Space Models

Alankrit Sinha
BITS, Pilani
h20250165@pilani-bits-pilani.ac.in

Abstract

Over the past few years, research on deep graph learning has shifted from static graphs to temporal graphs in response to real-world complex systems that exhibit dynamic behaviors. In practice, temporal graphs are formalized as an ordered sequence of static graph snapshots observed at discrete time points. Sequence models such as RNNs or Transformers have long been the predominant backbone networks for modeling such temporal graphs. Yet, despite the promising results, RNNs struggle with long-range dependencies, while transformers are burdened by quadratic computational complexity. Recently, state space models (SSMs), which are framed as discretized representations of an underlying continuous-time linear dynamical system, have garnered substantial attention and achieved breakthrough advancements in *independent* sequence modeling. In this work, we undertake a principled investigation that extends SSM theory to temporal graphs by integrating structural information into the online approximation objective via the adoption of a Laplacian regularization term. The emergent continuous-time system introduces novel algorithmic challenges, thereby necessitating our development of LTG-SSM, a graph state space model for modeling the dynamics of temporal graphs. Extensive experimental results demonstrate the effectiveness of our LTG-SSM framework across various temporal graph benchmarks.

1 Introduction

The goal of a sequence model is to map an input sequence to an output sequence, handling either continuous signals or discrete inputs. Historically, Recurrent Neural Networks (RNNs) [4, 3, 9] have dominated this space, offering theoretically infinite context windows and efficient $O(1)$ constant-time inference per token. However, RNNs suffer from vanishing and exploding gradients, making training strictly sequential ($O(N)$) and plagued by rapid forgetting on long sequences. More recently, Transformers [14] shifted the paradigm by allowing parallelized training. Yet, Transformers face their own limitation: the computational and memory complexity of self-attention is quadratically dependent on the sequence length ($O(N^2)$), severely restricting their application to very long sequences [**?**].

In an ideal world, a sequence model would combine the best of both architectures: parallelizable training that scales linearly like the Transformer, and constant $O(1)$ computation and memory per token during inference like the RNN. Recently, State Space Models (SSMs) [**?**, **?**, 12, **?**, **?**, 7] have emerged as precisely this powerful alternative. SSMs conceptualize sequential inputs as discrete observations from an underlying continuous-time linear dynamical system (LDS). A core component of modern SSMs is the High-order Polynomial Projection Operator (HiPPO) [**?**] theory. HiPPO mathematically builds the state transition matrix (A) to project all history seen so far into a compressed state vector using orthogonal Legendre polynomials. This allows the state to optimally memorize long-range histories while addressing the vanishing gradient issues of traditional differential equations.

While SSMs have achieved breakthrough advancements in independent, 1-dimensional sequence modeling, many real-world complex systems—such as social, traffic, and financial networks—exhibit

structural behaviors that are better represented as *temporal graphs* [10, **?**] (see figure **??** for an illustration). Temporal graphs manifest as discrete snapshots of an underlying network that continuously evolves over time. A primary motivation of our work is to reduce the prohibitively expensive $O(N^2)$ computations of Transformers when learning over these long graph sequences, specifically for the task of node classification. To model these graphs efficiently, sequence models must be generalized to non-euclidean data. However, directly applying SSMs to temporal graphs poses a significant challenge. SSMs are predominantly architected towards independent sequence modeling and lack a principled mechanism for incorporating time-varying topological structures. Specifically, it remains unexplored how the foundational methodology of discretized continuous-time LDS, guided by HiPPO, can be unified with spatial dependencies across nodes.

In this work, we advance the SSM methodology to encompass temporal graphs from the first principles. Rather than simply applying existing SSMs to graph sequences, we dive into the fundamental problem of online function approximation that underpins HiPPO. By solving a novel Laplacian-regularized online approximation objective, we derive a new formulation where the governing equations are directly influenced by the evolving graph structure. This essentially forces the memory representations of adjacent nodes to be smooth with respect to the graph topology, turning the graph into an effective structural denoiser during memory compression.

Furthermore, this continuous-time formulation introduces a novel algorithmic challenge specific to temporal graphs: *unobserved graph mutations*. Because we only observe the graph at discrete timestamps, the continuous edge additions and deletions between snapshots are hidden. To solve this, we propose a mixed discretization strategy that estimates this hidden evolution using feature mixing and graph neural networks (GNNs). This culminates in our design of LTG-SSM, a comprehensive state space framework for temporal graphs.

The main contributions of this work are summarized as follows:

- **The GHiPPO Abstraction:** We introduce GHiPPO, a novel construct predicated on a Laplacian-regularized online function approximation. This acts as a memory compression primitive that simultaneously compresses both the continuous feature dynamics and the evolving topological structure of the temporal graph.
- **The LTG-SSM Framework:** We design LTG-SSM, a flexible state space framework that addresses the unobserved graph mutation challenge. We introduce a mixed discretization strategy that incorporates an interpolation gating mechanism (to balance past and present features) alongside a shallow GNN (to simulate inter-node diffusion).
- **Extensive Evaluation:** Experimental results on six temporal graphs validate the effectiveness of LTG-SSM. Our framework demonstrates substantial scaling advantages over existing temporal GNNs, generalizing highly effectively to temporal graphs with exceptionally long-range snapshots.

2 Background: State Space Models

2.1 Differential Equations and Discretization

State space models (SSMs) allow us to map a 1-dimensional input signal $x(t) \in \mathbb{R}$ to an output signal $y(t) \in \mathbb{R}$ by means of a continuous-time latent state representation $h(t) \in \mathbb{R}^N$. This mapping is governed by a linear ordinary differential equation (ODE):

$$h'(t) = Ah(t) + Bx(t) \tag{1}$$

$$y(t) = Ch(t) \tag{2}$$

where $A \in \mathbb{R}^{N \times N}$ dictates the evolution of the system's state over time, and $B \in \mathbb{R}^{N \times 1}, C \in \mathbb{R}^{1 \times N}$ project the input and output respectively. This system is linear and time-invariant (LTI), meaning the matrices A, B, C are fixed and do not depend on time t.

To operate on discrete sequences (e.g., tokens in a language model or graph snapshots at specific timestamps), this continuous-time model must be discretized. Given a step size Δ, we can approximate the continuous differential equation using techniques like the zero-order hold (ZOH) or Euler's method. This yields a discrete recurrent formulation:

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t \tag{3}$$

$$y_t = Ch_t \tag{4}$$

where $\overline{A}$ and $\overline{B}$ are the discretized parameters of the model.

2.2 HiPPO: Optimal Memory via Legendre Polynomials

The performance of an SSM relies heavily on the A matrix, which dictates how historical information is captured and propagated forward. Naive initializations lead to exponential vanishing gradients on long sequences. The High-order Polynomial Projection Operator (HiPPO) [**?**] theory provides a mathematically optimal construction for A. Rather than memorizing the exact signal, HiPPO projects the continuous history of the input up to time t onto a basis of orthogonal *Legendre polynomials*. The state vector $h(t)$ stores the coefficients of these polynomials, effectively acting as an optimally compressed "sketch" of the history. This ensures that the state prioritizes more recent inputs while gracefully decaying older information, preventing the vanishing gradient problem.

2.3 Selective SSMs (Mamba)

Vanilla SSMs (like S4) excel at continuous signal processing but struggle with discrete information-dense data (like text) because their LTI nature prevents content-aware reasoning. For example, an LTI system cannot selectively ignore specific tokens based on their content because matrices A, B, and C are fixed. Selective SSMs, such as Mamba [7], address this by making the parameters B, C, and the step size Δ input-dependent functions of x_t. This enables the model to dynamically choose what to remember and what to discard. To counteract the loss of convolutional parallelizability that comes with time-varying parameters, Selective SSMs utilize hardware-aware *kernel fusion*, keeping intermediate operations within the GPU's ultra-fast SRAM to avoid I/O bottlenecks with the High Bandwidth Memory (HBM).

3 Related work

3.1 Temporal graph learning

A major branch of temporal graph learning methods consists of snapshot-based methods, which handle discrete-time temporal graphs by learning the temporal dependencies across a sequence of time-stamped graphs. Early works mainly focus on learning node representations by simulating temporal random walks [**?**] or modeling the triadic closure process [**?**] on multiple graph snapshots. These methods typically generate piecewise constant representations and may suffer from the staleness problem [10].

In recent years, the most established solution has been to combine sequence models (e.g., RNNs [4] and SNNs [**?**, 6]) with static GNNs to capture temporal dependencies and correlations between snapshots [1, **?**, **?**, **?**]. To better translate the success achieved on static graphs in both their design and training strategies, recent frameworks such as ROLAND [15] and its variants [**?**, **?**] have been proposed to repurpose static GNNs to temporal graphs. While Transformers [14] have also been explored to capture sequence dynamics via self-attention, their prohibitive $O(N^2)$ computational and memory cost fundamentally restricts their usage to short, shallow sequences.

3.2 SSMs on Graph-Structured Data

Given the promising properties of SSMs detailed in Section 2—namely fast $O(1)$ inference and highly parallelizable $O(N)$ training—there has been growing interest in expanding SSMs to non-euclidean domains. Recent concurrent works such as GraphMamba [2] have begun exploring selective SSMs directly on graph topologies, demonstrating that SSMs can be a powerful alternative to traditional attention mechanisms on static graphs. However, our work explicitly pioneers the use of the HiPPO framework paired with Laplacian regularization to solve the unobserved graph mutation problem inherent to *dynamic, temporal graphs*.

Architecture Comparison. The usual paradigms for designing sequence models over temporal graphs involve recurrence (RNNs [4, 3, 9]) or attention (Transformers [14]), which each come with tradeoffs. RNNs offer constant computation per time step during inference but suffer from slow sequential training and rapid forgetting. Transformers solve the training bottleneck but introduce

quadratically expensive $O(N^2)$ memory scaling. In contrast, SSMs provide a balanced, high-performance alternative suitable for extremely long sequences. A summary comparison of these architectures is illustrated in Table 1.

Table 1: Comparisons of different neural network architectures for sequence modeling.

	RNNs [4, 3, 9]	**Transformers [14]**	**SSMs (S6 [7])**
Training	Slow	Fast	Fast
Inference	Fast	Slow	Fast

4 The LTG-SSM framework

The primary motivation of our framework is the fact that discrete-time temporal graphs are sequential observations of an underlying temporal graph that evolves continuously. Adopting this functional viewpoint, we will first develop a piecewise recurrent memory update scheme in Section 4.1. This scheme optimally approximates the continuous-time evolution by extending the standard HiPPO theory to graph-structured inputs, culminating in our GHiPPO formulation. Then, in Section 4.2, we present the architecture of LTG-SSM, detailing how we tackle the critical challenge of unobserved graph mutations and build the end-to-end framework.

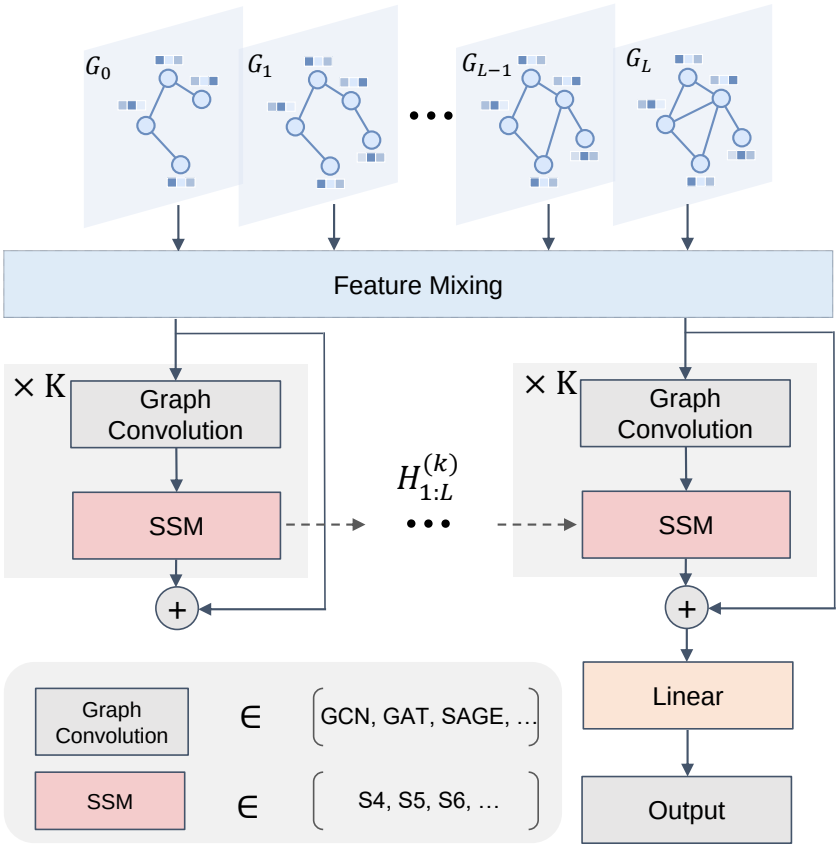

Figure 1: Conceptual Framework Depiction of Mamba structure.

4.1 From HiPPO to GHiPPO: Memory on Temporal Graphs

Setup. We formalize a temporal graph over a time interval $[0, T]$ as two interconnected processes: a graph process $G(t) = (V, E(t))$ and a node feature process $X(t) \in \mathbb{R}^{N_V \times d}$. We assume a fixed node set V with $N_V = |V|$ (extensions to varying node sets are discussed in Appendix **??**). While

the node features $X(t)$ vary continuously, the edge set $E(t)$ undergoes a finite number of discrete mutations (insertions or deletions) at specific timestamps, resulting in a piecewise-constant topology.

In practical scenarios, we only observe this continuous system at discrete timestamps $\tau_1, \ldots, \tau_L$ with time gaps $\Delta_l = \tau_l - \tau_{l-1}$. The observations form a sequence of snapshots $\{G_{1:L}, X_{1:L}\}$. Crucially, the actual structural mutations often occur *between* these observation windows and remain hidden.

The Laplacian-Regularized Objective. The goal of the standard HiPPO framework is to compress the continuous history of an input signal into a finite memory state $Z(t)$ by minimizing the approximation error over time. Standard HiPPO processes each dimension of the input independently. However, for temporal graphs, treating nodes independently ignores the rich structural information that dictates how signals propagate. For instance, in noisy dynamic systems, leveraging a node's neighborhood can effectively smooth and denoise the learned memory representations.

To integrate structural awareness directly into the memory compression process, we introduce a *Laplacian-regularized online approximation*. Assuming scalar features ($d = 1$) for simplicity, we simultaneously compress the history of all nodes $Z(t) \in \mathbb{R}^{N_V \times 1}$ by optimizing the following objective at time t:

$$\mathcal{L}_t(Z; G, X, \mu) = \int_0^t \|X(s) - Z(s)\|_2^2 \, d\mu_t(s) + \alpha \int_0^t Z(s)^\top L(s) Z(s) d\mu_t(s). \tag{5}$$

Here, $\alpha > 0$ controls the regularization strength, μ_t is a time-dependent measure weighting historical importance, and $L(s)$ is the normalized graph Laplacian at time s. The addition of the Laplacian term enforces a relational inductive bias: it forces the compressed memory of a node to remain smooth with respect to its topological neighbors over time.

To choose a suitable approximation subspace for memories Z, we adopt the approach of HiPPO that uses some N-dimensional subspace of polynomials which we denote as $\mathcal{P}_N$. Now we define a *graph memory projection operator* GPROJ_t that maps the temporal graph up until time t to a collection of N_V polynomials with each one lies in $\mathcal{P}_N$, i.e.,

$$\text{GPROJ}_t(G, X) = \underset{Z: z_v \in \mathcal{P}_N \ \forall v \in V}{\arg\min} \mathcal{L}_t(Z; G, X, \mu). \tag{6}$$

We further define a *coefficient* operator COEF_t that maps each polynomial in the collection in (6) to the coefficients of the basis of orthogonal polynomials defined with respect to μ_t, the following definition formalizes our extension of HiPPO to continuous-time temporal graphs which we term GHiPPO:

Definition 1 (GHiPPO)**.** *Given a continuous-time temporal graph (G, X), a time-varying measure family μ_t, an N-dimensional subspace of polynomials $\mathcal{P}_N$, the* GHiPPO *operator at time t is the composition of GPROJ_t and COEF_t that maps the temporal graph and node features to a collection of projection coefficients $U(t) \in \mathbb{R}^{N_V \times N}$, or* $\text{GHiPPO}(G, X) = \text{COEF}_t(\text{GPROJ}_t(G, X))$.

The most favorable property of the HiPPO framework on independent inputs is that the outputs of HiPPO operators are characterized via a concise ordinary differential equation (ODE) that takes the form of a linear time-invariant state space model (LTI-SSM). The following theorem states that most of the desirable properties of HiPPO are retained by GHiPPO except for the LTI property:

Theorem 1. *Let $G(t)$ evolve as a piecewise-constant topology. Taking μ_t to be the scaled Legendre measure (LegS) with $\mu_t = \frac{1}{t}\mathbb{I}_{[0,t]}$ where $\mathbb{I}_{[0,t]}$ stands for the indicator function of the interval $[0, t]$, the evolution of the outputs of* GHiPPO *operator is characterized by M ODEs according to mutation times as follows:*

$$\frac{dU(t)}{dt} = U(t)A^\top + (I + \alpha L(t))^{-1} X(t) B^\top, \quad 1 \le m \le M, t \in [t_{m-1}, t_m) \tag{7}$$

where $A \in \mathbb{R}^{N \times N}$ and $B \in \mathbb{R}^{N \times 1}$ takes the same form as in the HiPPO *formulation [**?**]:*

$$A_{nk} = -\begin{cases} \sqrt{(2n+1)(2k+1)} & \text{if } n > k, \\ n+1 & \text{if } n = k, \\ 0 & \text{if } n < k, \end{cases} \quad \text{and} \quad B_n = \sqrt{2n+1}, 1 \le n \le N. \tag{8}$$

According to theorem 1, the solution (7) is LTI over each interval $[t_{m-1}, t_m)$ during which the graph structure remains fixed. This property further extends to a piecewise LTI perspective over the interval

$[0, T]$. Moreover, we may view the solution (7) as a two-stage procedure that could be intuitively described as *diffuse-then-update*. Specifically, this procedure entails a sequential execution, wherein an initial diffusion operation is applied to the features of the input nodes, succeeded by an update to the memory of these nodes.

4.2 Architecture Overview

We propose **LTG-SSM**, a spatio-temporal architecture for dynamic graphs that integrates graph-based diffusion with state space models. The model operates on a sequence of discrete graph snapshots $\{(G_l, X_l)\}_{l=1}^{L}$ where G_l denotes the graph structure and $X_l \in \mathbb{R}^{N_V \times d}$ represents node features at time snapshot l.

4.2.1 Overview

Each LTG-SSM layer consists of four main components:

1. **Spatial Diffusion (GNN)**
2. **Temporal Feature Mixing**
3. **State Space Model (S4)**
4. **Residual Projection and Activation**

A key design choice in LTG-SSM is to **apply spatial diffusion before temporal mixing**, enabling temporal modeling in a **structure-aware latent space**. The complete end-to-end architecture is illustrated in Figure 2.

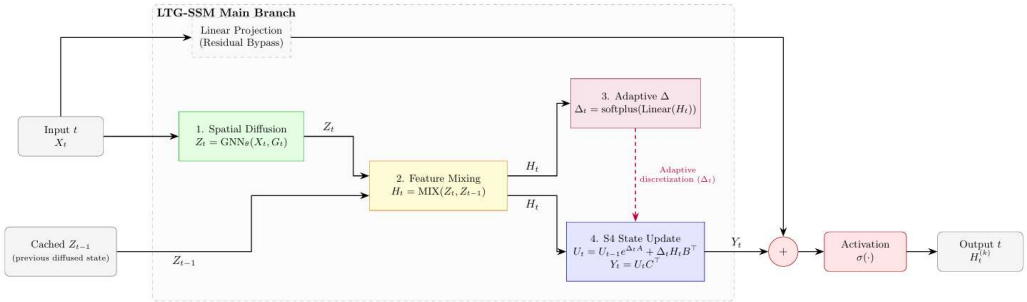

Figure 2: LTG-SSM architecture overview.

4.2.2 Spatial Diffusion

We first apply a graph neural network to propagate information across the observed graph:

$$Z_l = \mathrm{GNN}_\theta(X_l, G_l) \tag{9}$$

This operation acts as a **diffusion operator**, approximating the effect of latent structural evolution between observed snapshots. Instead of explicitly modeling discrete graph mutations, the GNN captures their aggregate influence through neighborhood aggregation.

4.2.3 Temporal Feature Mixing

To incorporate temporal dependencies and tackle unobserved topological mutations that occur between discrete snapshots, we mix representations from consecutive time steps:

$$H_l = \mathrm{MIX}_\phi(Z_l, Z_{l-1}) \tag{10}$$

The mixing function can be implemented via an INTERP (Interpolation) gating mechanism, using a sigmoid function to decide exactly how much to favor the past state versus the present state. Unlike prior approaches that mix raw features, LTG-SSM performs mixing in the **diffused representation space**, allowing temporal interactions to be conditioned on graph structure.

4.2.4 Adaptive State Space Model

Temporal dynamics are modeled using a structured state space model. Given the mixed features H_l, we compute an adaptive step size:

$$\Delta_l = \mathsf{softplus}(\textsc{Linear}(H_l)) \tag{11}$$

The hidden state evolves as:

$$U_l = U_{l-1} e^{\Delta_l A} + \Delta_l H_l B^\top \tag{12}$$

and the output is computed as:

$$Y_l = U_l C^\top \tag{13}$$

where A, B, C are learnable parameters. This formulation enables efficient modeling of long-range temporal dependencies with linear complexity. We utilize the diagonal state space approximation $A^{-1}(e^{\Delta A} - I) \approx \Delta I$ for computational efficiency.

4.2.5 Residual Projection and Output

To stabilize training and improve gradient flow, we incorporate a residual connection:

$$\widehat{Y}_l = Y_l + \textsc{Linear}(X_l) \tag{14}$$

The final output of the k-th block is obtained using a non-linear activation σ:

$$H_{1:L}^{(k)} = \sigma(\widehat{Y}_{1:L}) \tag{15}$$

where $H_{1:L}^{(0)} = X_{1:L}$. Similar to standard sequence modeling literature, the SSM layer can be instantiated to handle multi-dimensional node features by running an independent SISO SSM for each feature channel (**LTG-SSM-S**4). A detailed algorithmic exposition is provided in Appendix **??**.

4.2.6 Design Rationale

The proposed ordering—**diffusion before temporal mixing**—differs from prior approaches that perform mixing in the raw feature space. This design provides two key benefits:

- **Structure-aware temporal modeling:** Temporal interactions operate on graph-informed representations.
- **Improved robustness:** Early diffusion mitigates noise and partially observed structural changes.

Thus, LTG-SSM can be interpreted as performing **temporal evolution in a diffused latent space**, leading to more stable and expressive representations for dynamic graphs.

5 Experiments

This section presents our key experimental findings on the temporal node classification task. The detailed experimental setup is presented below.

5.1 Experimental setup

Datasets. We focus on the node classification task in discrete-time temporal graphs, which is a straightforward extension of static graphs. The experiments are conducted on two temporal graph benchmarks, including DBLP-3 [1] and Brain [1]. Dataset statistics are summarized in table 2. The graph datasets are collected from real-world networks belonging to different domains.

Table 2: Dataset Statistics.

	DBLP-3	Brain	Tmall
#Nodes	4,257	5,000	577,314
#Edges	23,540	1,955,488	4,807,545
#Features	100	20	128
#Classes	3	10	5
#Time Steps	10	12	186
Category	Citation	Biology	E-commerce

Baselines. We compare LTG-SSM with the following state-of-the-art baselines: (i) discrete-time temporal graph neural networks: STAR [1] and ROLAND [15]; (ii) temporal graph Transformers: DyGFormer [16]. All baselines are carefully tuned to achieve their best results based on the code officially provided by the authors.

Implementation details. LTG-SSM utilizes the S4 architecture. For handling unobserved graph mutations, we exclusively employ Feature Mixing via the MIX module for all datasets. The graph convolution networks used to learn the graph structure are SAGE [8] for all datasets, except for arXiv, where GCN [11] is used. All models are trained for a maximum of 50 epochs with early stopping applied based on validation performance. We implement our models as well as baselines with PyTorch [13] and PyTorch Geometric [5], which are open-source software released under BSD-style [1] and MIT [2] license, respectively. All datasets used throughout experiments are publicly available. All experiments are conducted on an Dell A100 GPU with 80 GB memory. Code will be made available at `https://github.com/KriT-S11/LTG-SSM`.

Evaluation protocol. We adopt the conventional *transductive* learning setting, where the graph structure of all snapshots is visible during both training and inference stages. This is analogous to the standard node classification task, but with the additional incorporation of time information to facilitate the learning. For the DBLP-3 and Brain datasets, we adopt the 81%/9%/10% train/validation/test splits as suggested in [1]. We use Micro-F1 and Macro-F1 to evaluate the node classification performance. We report the average performance with standard deviation across 5 runs for each method.

Table 3: Node classification performance (%) and training time (s/epoch) on DBLP-3, Brain, and Tmall temporal graphs. The best and the second best results are highlighted as **green** and **blue**, respectively.

	DBLP-3			Brain			Tmall		
	Micro-F1	**Macro-F1**	**Time (s)**	**Micro-F1**	**Macro-F1**	**Time (s)**	**Micro-F1**	**Macro-F1**	**Time (s)**
DyGFormer [16]	**85.61**$_{\pm 0.8}$	**85.42**$_{\pm 1.0}$	124.5	**93.88**$_{\pm 1.1}$	**93.91**$_{\pm 0.8}$	320.6	OOM	OOM	OOM
STAR [1]	84.74$_{\pm 1.0}$	84.20$_{\pm 1.2}$	75.4	92.08$_{\pm 1.3}$	92.23$_{\pm 1.3}$	172.5	OOM	OOM	OOM
ROLAND [15]	84.21$_{\pm 1.4}$	84.06$_{\pm 1.5}$	68.1	92.14$_{\pm 1.2}$	91.85$_{\pm 1.1}$	156.2	OOM	OOM	OOM
LTG-SSM	**85.26**$_{\pm 0.9}$	**85.00**$_{\pm 1.3}$	**26.8**	**94.52**$_{\pm 1.0}$	**94.54**$_{\pm 0.9}$	**50.4**	**66.29**$_{\pm 0.1}$	**62.57**$_{\pm 0.1}$	**145.3**

5.2 Experimental results

Node classification performance. The node classification performance is presented in table 3. It can be observed that LTG-SSM achieves highly competitive performance across the DBLP-3, Brain, and Tmall datasets, consistently outperforming advanced discrete-time GNNs like STAR and ROLAND. Notably, on the Brain dataset—which is characterized by a very small number of nodes but a dense, high number of edges—LTG-SSM consistently outperforms DyGFormer, achieving state-of-the-art results while being over $6\times$ faster. This demonstrates that by grounding our memory compression in the Laplacian-regularized ODE, LTG-SSM effectively denoises the structural topology, allowing it to approach or exceed the representational power of heavy Transformers without needing expensive pairwise attention. Furthermore, LTG-SSM consistently outperforms advanced

[1] `https://github.com/pytorch/pytorch/blob/master/LICENSE`

[2] `https://github.com/pyg-team/pytorch_geometric/blob/master/LICENSE`

discrete-time GNNs like STAR and ROLAND, verifying the effectiveness of our mixed discretization strategy for capturing unobserved graph mutations.

Scalability to large temporal graphs. To explore the effectiveness of LTG-SSM on large-scale and long-range temporal graphs, we conduct comparison experiments on the massive Tmall dataset. With over 577k nodes and 186 time steps, the graph scale and long sequence pose significant challenges for learning, leading most advanced baselines (including STAR, ROLAND, and DyGFormer) to encounter out-of-memory (OOM) failures. Specifically, even when utilizing an 80GB A100 GPU cluster, the $O(N^2)$ memory complexity of self-attention in Transformers and the massive activation bloat caused by Backpropagation Through Time (BPTT) in recurrent models rapidly exhaust the available VRAM. In contrast, by leveraging the linear $O(N)$ efficiency and sequence compression capability of SSMs, LTG-SSM bypasses these memory bottlenecks. LTG-SSM efficiently scales to this dataset and achieves strong classification performance (66.29% Micro-F1 / 62.57% Macro-F1). This underscores that LTG-SSM is uniquely equipped to efficiently capture long-range graph dynamics on large graphs where traditional methods fail.

Computational Efficiency. To validate our core motivation of challenging the computational bottlenecks of Transformers, we present the training time comparison alongside the performance metrics in table 3. LTG-SSM significantly outperforms both RNN/GNN and Transformer baselines in terms of training speed across all datasets. While Transformer architectures like DyGFormer suffer from a super-linear $O(L^2)$ complexity that restricts their scalability, RNN-based models like ROLAND are burdened by sequential unrolling. Because RNNs must process the graph step-by-step, they incur massive computational overhead at each snapshot. LTG-SSM avoids both pitfalls: by leveraging the associative property of the state space formulation, LTG-SSM achieves linear $O(L)$ complexity while remaining highly parallelizable over the time dimension, allowing it to train in a fraction of the time taken by competing models.

6 Conclusion

In this work, we introduce a conceptualized GH1PPO abstraction on temporal graphs. Building upon GH1PPO, we propose LTG-SSM, a theoretically motivated state space framework for modeling temporal graphs derived from a novel memory compression scheme. The proposed framework is computationally efficient and versatile in its design, which is further corroborated by strong empirical performance across various benchmark datasets. We also point out the unobserved graph mutation issue in temporal graphs and propose different mixing mechanisms to ensure temporal continuity across consecutive graph snapshots. Despite the promising results, the applicability of LTG-SSM is presently confined to discrete-time temporal graphs.

References

[1] Anonymous. Star: Spatial-temporal augmentation with routing network for spatial-temporal graph neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[2] A. Behrouz and F. Hashemi. Graph mamba: Towards learning on graphs with state space models. *CoRR*, abs/2402.08678, 2024.

[3] K. Cho, B. Van Merrinboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

[4] J. L. Elman. Finding structure in time. *Cognitive science*, 14(2):179–211, 1990.

[5] M. Fey and J. E. Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[6] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski. *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press, 2014.

[7] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. *CoRR*, abs/2312.00752, 2023.

[8] W. Hamilton, Z. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, volume 30, 2017.

[9] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

[10] S. M. Kazemi, R. Goel, K. Jain, I. Kobyzev, A. Sethi, P. Forsyth, and P. Poupart. Representation learning for dynamic graphs: A survey. *J. Mach. Learn. Res.*, 21(1), jan 2020.

[11] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[12] E. Nguyen, K. Goel, A. Gu, G. W. Downs, P. Shah, T. Dao, S. Baccus, and C. Ré. S4ND: modeling images and videos as multidimensional signals with state spaces. In *NeurIPS*, 2022.

[13] A. Paszke et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. 2019.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[15] J. You, T. Du, and J. Leskovec. Roland: graph learning framework for dynamic graphs. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 2358–2366, 2022.

[16] L. Yu, L. Sun, B. Du, C. Liu, H. Xiong, and W. Lv. Towards better dynamic graph learning: New architecture and unified library. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.